

Encapsulation in Object-Oriented Programming

1. Competence

After conducting experiments on this module, students understand the concept:

1. Constructor
2. Access Modifier
3. Attributes/methods in a class
4. Attribute/method intanization
5. Setter and getter
6. Understanding notation in a UML Class Diagram

2. Introduction

In the first and second meetings, the basic concepts of object-based programming (PBO), the difference between object-based programming and structural programming, were discussed, and the concepts of classes and objects in PBO were discussed. Furthermore, in this module, the concept of encapsulation in PBO and the notation in the UML Class diagram will be discussed.

2.1 Encapsulation

In the first module, the definition of encapsulation has been described as follows:

Encapsulation is also known as **information-hiding**. In interacting with objects, we often do not need to know the complexity that exists in them. This will be easier to understand if we imagine or analyze objects around us, for example bicycle objects, when we change gears on a bicycle, we just press the gear lever on the handlebar grip of the bicycle. We don't need to know how the gears move technically. Examples of other objects are for example a vacuum cleaner, when we plug in the vacuum cleaner cable and turn on the switch, the machine is ready to be used to suck dust. In this process, we do not know the complicated process that occurs when converting electricity into power from a vacuum cleaner. In the example above, the vacuum cleaner and bicycle have implemented encapsulation or also known as **information-hiding or data hiding** because it hides the details of the process of an object from the user.

2.2 Constructors

The constructor is similar to the method of declaration but does not have a return type. And the constructor is executed when an instant of the object is created. So every time an object is created with the keyword `new()` then the constructor will be executed. The way to create a constructor is as follows:

1. The constructor name must be the same as the class name
2. The constructor does not have a return data type
3. Constructors must not use abstract, static, final, and synchronized modifiers

Note: In java we can have constructors with private, protected, public or default modifiers.

In general, constructors are declared with the **public** modifier to ensure that the class can be freely instantiated by driver classes or other external components within the system. However, in advanced object-oriented programming architecture, using other modifiers is allowed to fulfill specific design needs:

1. **Private Constructor:** Used to completely restrict instantiation access from outside the class. This pattern is commonly applied in the implementation of the Singleton Pattern (ensuring only a single object exists in memory) or in Utility Classes that contain only a collection of static methods (such as Java's built-in **Math** class), making object creation for that class irrelevant.
2. **Default/Protected Constructor:** Used when object instantiation needs to be restricted to classes within the same package, or allowed to be initialized only by its subclasses within the corridor of inheritance.

2.3 Access Modifier

There are 2 types of modifiers in java, namely: access modifier and non-access modifier. In this case we'll focus on access modifiers that are useful for setting access methods, classes, and constructors. There are 4 access modifiers, namely:

1. **private** – can only be accessed within the same class
2. **default** – can only be accessed within the same package
3. **protected** – can be accessed outside the package using subclasses (create inheritance)
4. **public** – can be accessed from anywhere

Modifier access details can be seen in Table 1.1.

Table 1. 1 Detail Access Edit

Access Edit	Notation	within class	within package	outside package by subclass only	outside package
Private	(minus)	Y	N	N	N
Default	~ (tilde)	Y	Y	N	N
Protected	# (hash)	Y	Y	Y	N
Public	+ (plus)	Y	Y	Y	Y

2.4 Getter and Setter

Getter is a public method and has a data return type, which functions to get the value of the private attribute. Meanwhile, setter is a public method that does not have a data return type, which functions to manipulate the value of the private attribute.

2.5 UML Class Diagram Notation

In general the UML class diagram form is as shown in Figure 1.1

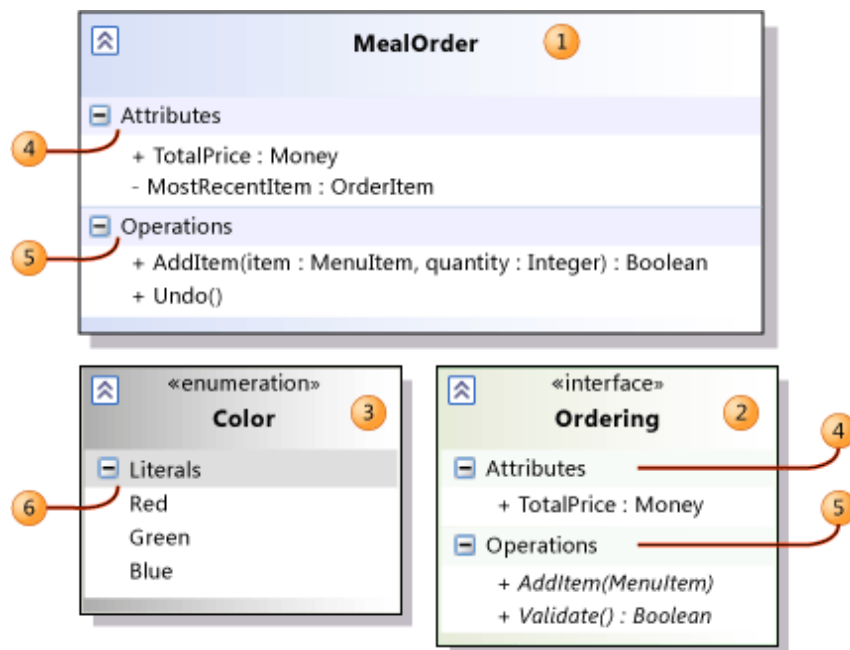


Figure 1. 1 UML Class Diagram

Information:

1. Class
2. Interface
3. Enumeration – is a type of data that has limited value or literality.
4. Attributes
5. Method
6. Literals

The access modifier notation in the UML class diagram is as follows:

1. Plus sign (+) for public
2. Hashtag (#) for protected
3. Minus sign (-) for private
4. Tilde sign (~) or unsign for default

3. Experiments

3.1 Experiment 1 - Encapsulation

In the encapsulation experiment, create a Motor class that has the speed and contactOn attributes, and has the printStatus() method to display the motor state. As follows

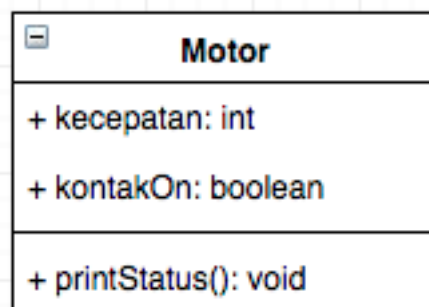
1. Open Netbeans, create a **MotorEncapsulation project**.
2. Create a **Motorcycle** class. Right-click on the package **motorencapsulation** – New – Java Class.
3. Type the Motorcycle class code below.

```

1  package motorencapsulation;
2
3  public class Motor {
4      public int kecepatan = 0;
5      public boolean kontakOn = false;
6
7      public void printStatus(){
8          if (kontakOn == true){
9              System.out.println("Kontak On");
10         }
11         else{
12             System.out.println("Kontak Off");
13         }
14         System.out.println("Kecepatan " + kecepatan+"\n");
15     }
16 }

```

The UML class diagram form of the Motor class diagram is as follows:



4. Then create a MotorDemo class, type in the following code.

```

1  package motorencapsulation;
2
3  public class MotorDemo {
4      public static void main(String[] args) {
5          Motor motor = new Motor();
6          motor.printStatus();
7          motor.kecepatan = 50;
8          motor.printStatus();
9      }
10 }

```

5. The results are as follows:

```
run:
Kontak Off
Kecepatan 0

Kontak Off
Kecepatan 50

BUILD SUCCESSFUL (total time: 0 seconds)
```

From experiment 1 - encapsulation, do you think anything is wrong?

That is, the speed of the motor suddenly changes from 0 to 50. Even more strange, the motorcycle contact position is still in the OFF condition. How is it possible for a motorcycle to speed from zero to 50 instantly, and even then the ignition is OFF?

Now in this case, access to the motorcycle's attributes turned out to be uncontrollable. In fact, objects in the real world always have limitations and mechanisms for how they can be used. Then, how can we improve the Motorcycle class above so that it can be used properly? We can consider the following:

1. Hide internal attributes (speed, contactOn) from users (other classes)
2. Provides a special method for accessing attributes.

For that, let's continue the following experiment about Access Modifier.

3.2 Experiment 2 - Access Modifier

In this experiment, an access modifier will be used to improve the way the Motor class works in the 1st experiment.

1. Change the way the motor class works according to the following UML class diagram.



2. Based on the UML class diagram, there are changes in the Motor class, namely:
 - a. Change access modifier speed and kontakOn to private
 - b. Add the method of startingEngine, turning offEngine, addSpeed, DecreaseSpeed.

The implementation of the Motor class is as follows:

```

1 package motorencapsulation;
2 public class Motor {
3     private int kecepatan = 0;
4     private boolean kontakOn = false;
5     public void nyalakanMesin(){
6         kontakOn = true;
7     }
8     public void matikanMesin(){
9         kontakOn = false;
10        kecepatan = 0;
11    }
12    public void tambahKecepatan(){
13        if (kontakOn == true){
14            kecepatan += 5;
15        }
16        else{
17            System.out.println("Kecepatan tidak bisa bertambah karena Mesin Off! \n");
18        }
19    }
20    public void kurangiKecepatan(){
21        if (kontakOn == true){
22            kecepatan -= 5;
23        }
24        else{
25            System.out.println("Kecepatan tidak bisa berkurang karena Mesin Off! \n");
26        }
27    }
28    public void printStatus(){
29        if (kontakOn == true){
30            System.out.println("Kontak On");
31        }
32        else{
33            System.out.println("Kontak Off");
34        }
35        System.out.println("Kecepatan " + kecepatan+"\n");
36    }
37 }

```

access modifier diubah private

method yang ditambahkan

3. Then in the MotorDemo class, change the code to the following:

```

1 package motorencapsulation;
2
3 public class MotorDemo {
4     public static void main(String[] args) {
5         Motor motor = new Motor();
6         motor.printStatus();
7         motor.tambahKecepatan();
8
9         motor.nyalakanMesin();
10        motor.printStatus();
11
12        motor.tambahKecepatan();
13        motor.printStatus();
14
15        motor.tambahKecepatan();
16        motor.printStatus();
17
18        motor.tambahKecepatan();
19        motor.printStatus();
20
21        motor.matikanMesin();
22        motor.printStatus();
23    }
24 }

```

4. The results of the MotorDemo class are as follows:

```
run:
Kontak Off
Kecepatan 0
Kecepatan tidak bisa bertambah karena Mesin Off!
Kontak On
Kecepatan 0
Kontak On
Kecepatan 5
Kontak On
Kecepatan 10
Kontak On
Kecepatan 15
Kontak Off
Kecepatan 0
BUILD SUCCESSFUL (total time: 0 seconds)
```

From the above experiment, we can observe that now the speed attribute cannot be accessed by the user and is replaced with a value carelessly. Even when trying to increase speed while the contact position is still OFF, a notification will appear that the engine is OFF. To get the desired speed, it must be done gradually, namely by calling the `addSpeed()` method several times. This is similar to when we ride a motorcycle.

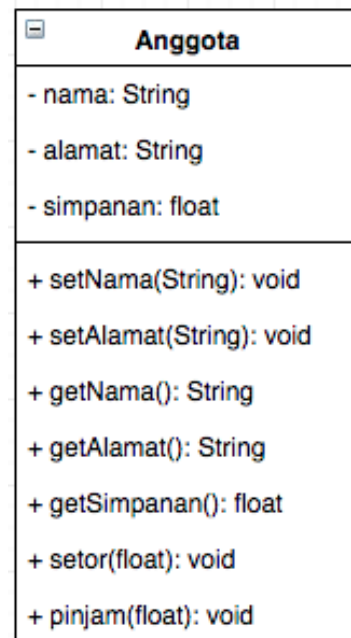
3.3 Questions

1. In the `TestMobil` class, when we increase the speed for the first time, why does the warning "Speed cannot increase due to Engine Off!"?
2. Connecting the speed attribute and `contactOn` set private?
3. Change the `Bike` class so that the maximum speed is 100!

3.4 Experiment 3 - Getter and Setter

For example, in a cooperative information system, there is a class of `Members`. `Members` have the attributes of name, address and deposit, and method setter, getter and deposit and borrow. All attributes on the member should not be changed indiscriminately, but can only be changed through the method setter, getter, deposit and draw. Especially for the deposit attribute, there is no setter because the deposit will increase when making deposit transactions and will decrease when lending/withdrawing.

1. The following UML class makes a `Student` class in the program:



2. Same as experiment 1 to create a new project
 - a. Open Netbeans, create a **GetterSetter Cooperative** project.
 - b. Create a **Member** class. Right-click on the **package koperasi gettersetter** – New – Java Class.
 - c. Type in the Member class code below.

```

1 package kopersigettersetter;
2 public class Anggota {
3     private String nama;
4     private String alamat;
5     private float simpanan;
6
7     public void setNama(String nama){
8         this.nama = nama;
9     }
10    public void setAlamat(String alamat){
11        this.alamat = alamat;
12    }
13    public String getNama(){
14        return nama;
15    }
16    public String getAlamat(){
17        return alamat;
18    }
19    public float getSimpanan(){
20        return simpanan;
21    }
22    public void setor(float uang){
23        simpanan +=uang;
24    }
25    public void pinjam(float uang){
26        simpanan -=uang;
27    }
28 }

```

setter, getter
nama dan alamat

getter
simpanan

If you look at the Member class, the name and address attributes have 1 getter and a setter each. Meanwhile, the deposit attribute only has getDeposit(), because like the initial purpose, the deposit attribute will change its value if you make a deposit() and borrow/draw() transaction.

3. Next, create a Demo Cooperative class to try the Member class.

```

1 package kopersigettersetter;
2 public class KoperasiDemo {
3     public static void main(String[] args) {
4         Anggota anggota1 = new Anggota();
5         anggota1.setNama("Iwan Setiawan");
6         anggota1.setAlamat("Jalan Sukarno Hatta no 10");
7         anggota1.setor(100000);
8         System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
9
10        anggota1.pinjam(5000);
11        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
12    }
13 }

```

4. The results of the main method in the third step are

```
run:
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

As can be seen from the results of the experiment above, to change the deposit is not done directly by changing the deposit attribute, but through the deposit() and loan() methods. To display the name must also be via the getName() method, and to display the save via getSave().

3.5 Experiment 4 - Constructors, Institutions

1. The first step of experiment 4 is to change the KooperativDemo class as follows

```
1 package koperasigettersetter;
2 public class KooperativDemo {
3     public static void main(String[] args) {
4         Anggota anggota1 = new Anggota();
5         System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
6
7         anggota1.setNama("Iwan Setiawan");
8         anggota1.setAlamat("Jalan Sukarno Hatta no 10");
9         anggota1.setor(100000);
10        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
11
12        anggota1.pinjam(5000);
13        System.out.println("Simpanan " + anggota1.getNama() + " : Rp " + anggota1.getSimpanan());
14    }
15 }
```

2. The results of the program are as follows

```
run:
Simpanan null : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

It can be seen that when the getName() method is called, the result is that this happens because the name attribute has not been set to the default value. This can be dealt with by making a constructor.

3. Change the Member class to the following

```
1 package kopersigettersetter;
2 public class Anggota {
3     private String nama;
4     private String alamat;
5     private float simpanan;
6
7     Anggota(String nama, String alamat){
8         this.nama = nama;
9         this.alamat = alamat;
10        this.simpanan = 0;
11    }
12
13    public void setName(String nama){
14        this.nama = nama;
15    }
16    public void setAddress(String alamat){
17        this.alamat = alamat;
18    }
19    public String getName(){
20        return nama;
21    }
22    public String getAddress(){
23        return alamat;
24    }
25    public float getSimpanan(){
26        return simpanan;
27    }
28    public void setor(float uang){
29        simpanan +=uang;
30    }
31    public void pinjam(float uang){
32        simpanan -=uang;
33    }
34 }
35
```

In the Member class, a constructor is created with a default access modifier that has 2 name and address parameters. And in the constructor, it is confirmed that the deposit value for the first time is Rp. 0.

4. Next, change the CooperativeDemo class as follows

```
1 package kopersigettersetter;
2 public class KoperasiDemo {
3     public static void main(String[] args) {
4         Anggota anggota1 = new Anggota("Iwan", "Jalan Mawar");
5         System.out.println("Simpanan " + anggota1.getName() + " : Rp " + anggota1.getSimpanan());
6
7         anggota1.setName("Iwan Setiawan");
8         anggota1.setAddress("Jalan Sukarno Hatta no 10");
9         anggota1.setor(100000);
10        System.out.println("Simpanan " + anggota1.getName() + " : Rp " + anggota1.getSimpanan());
11
12        anggota1.pinjam(5000);
13        System.out.println("Simpanan " + anggota1.getName() + " : Rp " + anggota1.getSimpanan());
14    }
15 }
```

Passing Parameters

5. The results of the program are as follows

```
run:
Simpanan Iwan : Rp 0.0
Simpanan Iwan Setiawan : Rp 100000.0
Simpanan Iwan Setiawan : Rp 95000.0
BUILD SUCCESSFUL (total time: 0 seconds)
```

After adding a constructor to the Anggoata class, the name and address attributes must automatically be set first by passing parameters if institutionalizing the Member class. This is usually done for attributes that require specific values. If it doesn't require a specific value in the constructor, it doesn't need a parameter. For example, the save for new members is set to 0, so the save does not need to be used as a parameter in the constructor.

3.6 Questions – Experiments 3 and 4

1. What do getters and setters mean?
2. What is the use of the getSave() method?
3. What method is used to top up the balance?
4. What does yand mean constructor?
5. What are the rules for creating constructors?
6. Is it OK to construct a private type?
7. When to use parameters with passing parameters?
8. What is the difference between a class attribute and an attribute instance?
9. What is the difference between a class method and an instance method?

4. Conclusion

From the experiment above, the concept of encapsulation, constructors, and access modifiers has been studied which consists of 4 types, namely public, protected, default and private. The concept of attributes or method classes that exist in the block of code class and the concept of attribute or method institutionalization. How to use getter and setter along with the functions of getter and setter. And have also learned or understood UML notation

5. Tasks

1. Try the program below and write down the output

```

public class EncapDemo
{
    private String name;
    private int age;

    public String getName()
    {
        return name;
    }

    public void setName(String newName)
    {
        name = newName;
    }

    public int getAge()
    {
        return age;
    }

    public void setAge(int newAge)
    {
        if(newAge > 30)
        {
            age = 30;
        }
        else
        {
            age = newAge;
        }
    }
}

public class EncapTest
{
    public static void main(String args[])
    {
        EncapDemo encap = new EncapDemo();
        encap.setName("James");
        encap.setAge(35);

        System.out.println("Name : " + encap.getName());
        System.out.println("Age : " + encap.getAge());
    }
}

```

2. In the program above, in the EncapTest class we set the age to 35, but when it is displayed on the screen the value is 30, explain why.

Answer:

3. Change the program above so that the age attribute can be given a maximum value of 30 and a minimum of 18.

Answer:

4. In an expedition cargo warehouse management system, there is a **Kontainer** class with attributes including **nomorResi**, **namaPemilik**, **kapasitasMaksimal** (in kg), and **beratMuatanSaatIni**. A container can accept additional cargo weight up to its specified maximum capacity limit. Cargo can also be unloaded from the container. When cargo is unloaded, the current load weight will decrease according to the weight removed.

Create the **Kontainer** class with attributes (**private**), getter methods, and constructors according to encapsulation architecture requirements. Test it with the following **TestLogistik** driver class:

```

1 public class TestLogistik {
2     public static void main(String[] args) {
3         Kontainer kontainerAlfa = new Kontainer("REQ-9988", "PT. Maju Bersama", 5000);
4
5         System.out.println("Nama Pemilik Kontainer: " + kontainerAlfa.getNamaPemilik());
6         System.out.println("Kapasitas Maksimal: " + kontainerAlfa.getKapasitasMaksimal() + " kg");
7
8         System.out.println(x: x: "\nMemasukkan muatan baru seberat 6.000 kg...");
9         kontainerAlfa.tambahMuatan(6000);
10        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
11
12        System.out.println(x: x: "\nMemasukkan muatan baru seberat 4.000 kg...");
13        kontainerAlfa.tambahMuatan(4000);
14        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
15
16        System.out.println(x: x: "\nMembongkar muat/menurunkan barang seberat 500 kg...");
17        kontainerAlfa.turunkanMuatan(500);
18        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
19
20        System.out.println(x: x: "\nMembongkar muat/menurunkan barang seberat 1.500 kg...");
21        kontainerAlfa.turunkanMuatan(1500);
22        System.out.println("Berat muatan saat ini: " + kontainerAlfa.getBeratMuatanSaatIni() + " kg");
23    }
24 }

```

Expected results:

```

Nama Pemilik Kontainer: PT. Maju Bersama
Kapasitas Maksimal: 5000.0 kg

Memasukkan muatan baru seberat 6.000 kg...
Maaf, berat muatan melebihi kapasitas maksimal kontainer.
Berat muatan saat ini: 0.0 kg

Memasukkan muatan baru seberat 4.000 kg...
Berat muatan saat ini: 4000.0 kg

Membongkar muat/menurunkan barang seberat 500 kg...
Berat muatan saat ini: 3500.0 kg

Membongkar muat/menurunkan barang seberat 1.500 kg...
Berat muatan saat ini: 2000.0 kg

```

Answer:

- Modify the cargo logistics problem above so that the weight of the load unloaded in a single call to the `turunkanMuatan()` method is allowed to be a maximum of only 50% of the current total load weight. This measure is implemented for operational safety reasons regarding heavy equipment (cranes). If an operator attempts to unload cargo exceeding this 50% limit, the system must block the action and display a warning: "Sorry, for safety reasons, a single cargo unloading operation cannot exceed 50% of the current load!".

Answer:

6. Modify the driver class `TestLogistik` so that the weight parameter values entered for adding cargo (`tambahMuatan`) and unloading cargo (`turunkanMuatan`) can accept dynamic user inputs interactively through the terminal using the `java.util.Scanner` utility.

Answer:

7. A movie ticket booking application requires a `Tiket` class to manage booking data securely. This class must have the following private attributes: `judulFilm` (String), `hargaDasar` (double), and `statusPembayaran` (boolean).

Object Value Setup Requirements:

- The constructor must accept parameters for `judulFilm` and `hargaDasar`. The initial value of `statusPembayaran` is always set to `false` (Unpaid).
- The `hargaDasar` attribute cannot be negative. If the provided input is less than 0, automatically set it to a default value of Rp 35,000.
- Provide a `lakukanPembayaran()` method to change the `statusPembayaran` to `true`.
- The `statusPembayaran` value must be read-only using a getter and should not have a direct setter method from outside the class for transaction security reasons.

Test your code using the following `TestBioskop` class:

```
1 public class TestBioskop {
2     public static void main(String[] args) {
3         Tiket tiket1 = new Tiket("Avengers: Endgame", -50000);
4         System.out.println("Film: " + tiket1.getJudulFilm());
5         System.out.println("Harga Tiket: " + tiket1.getHargaDasar());
6         System.out.println("Status Lunas? " + tiket1.isStatusPembayaran());
7
8         System.out.println("\nMemproses pembayaran...");
9         tiket1.lakukanPembayaran();
10        System.out.println("Status Lunas Terbaru? " + tiket1.isStatusPembayaran());
11    }
12 }
```

Answer:

